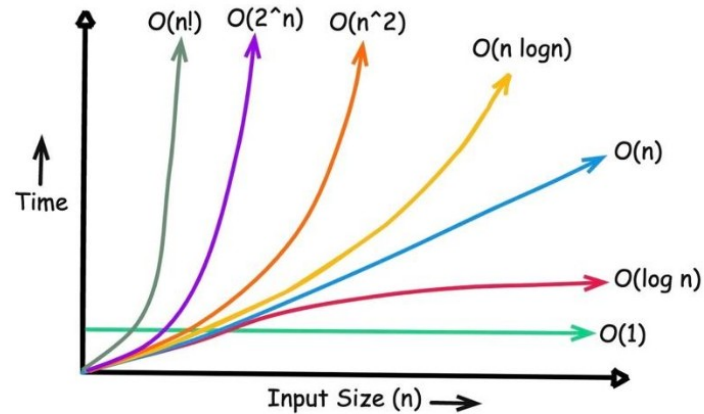


Lecture 6

Theoretical Running Time



Announcements

- ❖ **Office hours** are up on course website
 - T: 12pm-4pm; F: 11am-1pm
 - Schedule an appointment individually or as a group
- ❖ **Lab 1** due this Friday @ 11:59pm
- ❖ **Project 1** Planning & Analysis due **next Friday** @ 11:59pm
 - Details up now on course website
 - Choose a problem and [join a team](#) if you haven't
 - Be sure to comment your Github link on the project document

Today

- ❖ Counting steps
- ❖ Big-O Notation

Measuring Running Time

Two strategies:

- ❖ **Empirical**

- For a fixed compute environment, run an implementation of an algorithm on several inputs of varying properties and record execution time

- ❖ **Theoretical**

- Counts the number of primitive operations required in the worst case (up to multiplicative and additive constants)

Counting Steps

Why measure running time by counting steps?

Counting Steps

Why measure running time by counting steps?

- ❖ Gives us a universal way of discussing algorithms
- ❖ Abstracts away from implementation and hardware details



Counting Steps

Why measure running time by counting steps?

- ❖ Gives us a universal way of discussing algorithms
- ❖ Abstracts away from implementation and hardware details

What is a step?

- ❖ Basic instructions such as $+$, $-$, $*$, $=$, `if`
- ❖ Accessing an item in memory



Counting Steps

Why measure running time by counting steps?

- ❖ Gives us a universal way of discussing algorithms
- ❖ Abstracts away from implementation and hardware details

What is a step?

- ❖ Basic instructions such as `+`, `-`, `*`, `=`, `if`
- ❖ Accessing an item in memory

We measure running time as a function $T(n)$ of input size n



Example

Input: array A

Output: sum

```
sum = 0
```

```
n = length of A
```

```
for i = 1 to n do
```

```
    for j = 1 to n do
```

```
        sum += A[i] + A[j]
```

Example

Input: array A

Output: sum

```
sum = 0
```

```
n = length of A
```

```
for i = 1 to n do
```

```
    for j = 1 to n do
```

```
        sum += A[i] + A[j]
```

5 steps

Example

Input: array A

Output: sum

```
sum = 0
```

```
n = length of A
```

```
for i = 1 to n do
```

```
  for j = 1 to n do
```

```
    sum += A[i] + A[j]
```

5n steps

5 steps

Example

Input: array A

Output: sum

sum = 0

n = length of A

for i = 1 to n **do**

for j = 1 to n **do**

sum += A[i] + A[j]

$5n^2$ steps

$5n$ steps

5 steps

Example

Input: array A

Output: sum

sum = 0

n = length of A

2 steps

for i = 1 to n **do**

for j = 1 to n **do**

sum += A[i] + A[j]

$5n^2$ steps

5n steps

5 steps

Example

Input: array A

Output: sum

sum = 0

n = length of A

2 steps

for i = 1 to n **do**

for j = 1 to n **do**

sum += A[i] + A[j]

5n² steps

5n steps

5 steps

$$T(n) = 5n^2 + 2$$

Big-O Definition

Let $T(n)$ be the number of steps required for our algorithm in the worst case

Definition: We say that $T(n)$ is $O(f(n))$ if there exists a $c > 0$ such that

$$T(n) \leq cf(n)$$

for sufficiently large n

Big-O Definition

Let $T(n)$ be the number of steps required for our algorithm in the worst case

Definition: We say that $T(n)$ is $O(f(n))$ if there exists a $c > 0$ such that

$$T(n) \leq cf(n)$$

for sufficiently large n

- ❖ Then f is an **asymptotic upper bound** of T

Big-O Definition

Let $T(n)$ be the number of steps required for our algorithm in the worst case

Definition: We say that $T(n)$ is $O(f(n))$ if there exists a $c > 0$ such that

$$T(n) \leq cf(n)$$

for sufficiently large n

- ❖ Then f is an **asymptotic upper bound** of T
- ❖ Example: $T(n) = 2n^2 + n + 2$

Big-O Definition

Let $T(n)$ be the number of steps required for our algorithm in the worst case

Definition: We say that $T(n)$ is $O(f(n))$ if there exists a $c > 0$ such that

$$T(n) \leq cf(n)$$

for sufficiently large n

- ❖ Then f is an **asymptotic upper bound** of T
- ❖ Example:
$$\begin{aligned} T(n) &= 2n^2 + n + 2 \\ &\leq 2n^2 + n^2 + n^2 \text{ for } n \geq 2 \\ &= 5n^2 \end{aligned}$$

Big-O Definition

Let $T(n)$ be the number of steps required for our algorithm in the worst case

Definition: We say that $T(n)$ is $O(f(n))$ if there exists a $c > 0$ such that

$$T(n) \leq cf(n)$$

for sufficiently large n

- ❖ Then f is an **asymptotic upper bound** of T
- ❖ Example:
$$\begin{aligned} T(n) &= 2n^2 + n + 2 \\ &\leq 2n^2 + n^2 + n^2 \text{ for } n \geq 2 \\ &= 5n^2 \end{aligned}$$

Therefore, $T(n)$ is $O(n^2)$

What does Big-O mean?

Worst case analysis

- ❖ Running time guarantee for any input
- ❖ Typically captures the notion of *computational complexity*

Alternatives

- ❖ Average-case analysis
- ❖ Expected running time of a randomized algorithm
- ❖ Amortized (spread out over a sequence of operations)

} Too unwieldy or not
general enough

Common Bounds

- ❖ Constant time: $O(1)$
- ❖ Log time: $O(\log n)$
- ❖ Linear time: $O(n)$
- ❖ Log Linear time: $O(n \log n)$
- ❖ Quadratic time: $O(n^2)$
- ❖ Cubic time: $O(n^3)$
- ❖ Polynomial time: $O(n^k)$ for some $k > 0$
- ❖ Exponential time: $O(2^n)$

Common Bounds

- ❖ Constant time: $O(1)$
- ❖ Log time: $O(\log n)$
- ❖ Linear time: $O(n)$
- ❖ Log Linear time: $O(n \log n)$
- ❖ Quadratic time: $O(n^2)$
- ❖ Cubic time: $O(n^3)$
- ❖ Polynomial time: $O(n^k)$ for some $k > 0$
- ❖ Exponential time: $O(2^n)$

Faster



Slower

Common Bounds

- ❖ Constant time: $O(1)$
- ❖ Log time: $O(\log n)$
- ❖ Linear time: $O(n)$
- ❖ Log Linear time: $O(n \log n)$
- ❖ Quadratic time: $O(n^2)$
- ❖ Cubic time: $O(n^3)$
- ❖ Polynomial time: $O(n^k)$ for some $k > 0$
- ❖ Exponential time: $O(2^n)$

Computationally
feasible

Faster

Slower

Examples

- ❖ If $T(n) = n^2 + 10000000n$, then $T(n)$ is $O(n^2)$

Examples

❖ If $T(n) = n^2 + 1000000n$, then $T(n)$ is $O(n^2)$

❖ If $T(n) = n \log n + n^3$, then $T(n)$ is $O(n^3)$

Examples

- ❖ If $T(n) = n^2 + 1000000n$, then $T(n)$ is $O(n^2)$
- ❖ If $T(n) = n \log n + n^3$, then $T(n)$ is $O(n^3)$
- ❖ If $T(n) = n + 6$, then $T(n)$ is **not** $O(1)$

Next Time

- ❖ Properties of Asymptotic Upper Bounds
- ❖ How to use Big-O in practice
- ❖ Examples
- ❖ DSP 5.4-5.10